

Metal NAM Gear Players — Neo Edition — Guida Tecnica

Versione 0.2.0 — 120 parametri APVTS

Documentazione tecnica per utenti avanzati, tone-designer e sviluppatori.

Architettura

Host (VST3 / LV2 / Standalone)

```
└─ NAMCustomAudioProcessor (juce/PluginProcessor.*)
    └─ APVTS (parameters: input, output, ir_*, ng_*, gate_*, od_*, dist_*,
        eq_*, depth, reson, hp_*, ln_*, delay_*, chorus_*, flanger_*,
        reverb_*, tremolo_*, output_mode, calibrate_input,
        input_cal_level, quality_scale, os_2x, ...)
    └─ juce::dsp::Oversampling (opzionale 2x, IIR half-band polyphase)
    └─ Left/Right NAMPipeline (juce/NAMPipeline.*)
        └─ SmartGate / Compressore / OD / Dist (PreampFX.h)
        └─ NeuralAudio::NeuralModel (V1/A2) + volume e tone stack
        └─ Amp Sim nativo (GEAR SX / MARCHELLOW, opzionale)
        └─ EQ (src/dsp/EQ.h) + Depth + Resonance + NoiseGate + HP + LN
        └─ presa per l'analizzatore di spettro (coda circolare 4096)
        └─ IRConvolver x2 (FFT convolution) con bilanciamento
        └─ Delay / Chorus / Flanger / Reverb / Tremolo (PreampFX.h)
    └─ Metronome (juce/Metronome.h) mescolato a valle, frequenza base
    └─ meter: ingresso, uscita, lettore NAM, IR 1, IR 2, compressore
    └─ Async loader (ThreadPool per model/IR con swap atomico non bloccante)
```

Real-time safety

- **Hot path (processBlock) alloc-free:** allocazioni ammesse solo in prepare() e nei costruttori.
- **Load asincrono:** loadModelAsync / loadIRAsync fanno il parsing su un juce::ThreadPool e pubblicano il nuovo NAMPipeline in pendingL_/pendingR_ (raw ptr atomico). processBlock consuma lo swap non bloccante all'inizio del blocco.
- **Clear e prepare:** clearModel/clearIR e setOversamplingEnabled chiamano suspendProcessing(true/false) per evitare use-after-free con l'audio thread.
- **Modelli senza metadata:** la normalizzazione/calibrazione consulta i flag HasLoudness/HasInputLevel/HasOutputLevel esposti dal fork fabionet/NeuralAudio (branch nam-custom-patches); se assenti, nessuna correzione viene applicata.

Formato preset

Un preset puo' puntare a un file locale qualunque. C'era un elenco di cartelle consentite — home, Documenti, Musica, dati applicazione, temporanei — e tutto il resto veniva scartato **in silenzio**: modello e IR si azzeravano e il preset si apriva pulito senza dire perche'. Chi tiene la libreria su un disco esterno montato sotto /media, o su un secondo disco in Windows, non poteva usare i propri preset. La guardia che serve davvero e' rimasta ed e' una sola, NAMAudioProcessor::isLocalSafePath: rifiuta i percorsi UNC, quelli di device NT e gli schemi di rete, che sono la via del furto di credenziali. Un file locale qualunque, al massimo, non si lascia leggere come modello.

I caricatori seguono anche i cambi che non vengono da loro. Le liste si aggiornavano solo passando dal pulsante Browse: caricando un preset il modello e l'IR entravano davvero in catena, ma le caselle continuavano a dire "- none -" e sembrava che non si fosse caricato niente. Ora il timer dell'editor confronta i percorsi correnti del processore con gli ultimi visti e rilegge la cartella solo quando cambiano: la scansione e' ricorsiva e non va fatta a ogni giro.

Al salvataggio si scrivono solo i parametri che esistono. replaceState sostituisce l'albero intero e i nodi dei parametri spariti restano dentro, per poi essere riscritti al salvataggio successivo: un preset si portava dietro trenta voci morte e cresceva a ogni giro.

I modelli si salvano per nome, non per numero. Le sezioni a pedale e il selettore degli amplificatori scelgono con un indice dentro il loro registro, e quell'indice cambia ogni volta che si inserisce un modello in mezzo all'elenco. E' successo: dopo aver aggiunto tre overdrive in testa alla lista, tutti i preset di fabbrica si sono ritrovati un overdrive nella sezione dei distorsori e un pedale a caso in quella dei gate, senza che niente lo segnalasse. Da allora il preset porta anche un blocco `<Models>` con l'id testuale di ogni sezione, e `applyModelIds()` lo rilegge **dopo** `replaceState`: gli id non si muovono, gli indici si'. Un modello sparito lascia la sezione com'era invece di puntare a caso, e un preset scritto prima di questa modifica si carica lo stesso, solo senza la correzione.

Ogni variante di banco porta il proprio blocco: una variante puo' avere pedali diversi dal preset che la contiene.

`.nampreset` è XML (APVTS state) con l'aggiunta di:

- `<NAMPreset name="..." version="1" lockModel="0|1" category="...">`
- `<ModelPath>`, `<IRPath>` e `<IR2Path>` — percorso assoluto, oppure **nome nudo** di una risorsa incorporata nel binario, risolta da `resolveBundled` (che rifiuta separatori e `..`)
- `<Parameters>` con lo stato APVTS completo

I preset devono contenere tutti i parametri. `applyXml` usa `apvts_.replaceState()`, e i parametri assenti dall'albero **non tornano al default**: restano al valore corrente. Un preset parziale quindi non definisce del tutto il suono. Dopo ogni aggiunta di parametri vanno rigenerati i preset di fabbrica — sono passati da 62 a 111, poi 115, ora **120**.

Formati di scambio

Estensione	Contenuto
<code>.nampreset</code>	formato nativo, un <code><NAMPreset></code>
<code>.prs</code>	preset singolo da esportazione
<code>.prstl</code>	elenco di preset, con gli asset <code>.nam</code> e <code>.wav</code> incorporati in base64

L'importazione accetta **solo** queste tre estensioni e pretende comunque che la radice sia un `<NAMPreset>` o lo contenga. In un `.prstl` ogni asset è incorporato una volta sola anche se più preset lo usano; al ripristino i file vengono scritti in `~/ .config/NAMCustom/Assets` e i riferimenti riscritti. I nomi con separatori o `..` vengono scartati.

I preset di fabbrica sono embedded nel binario tramite `juce_add_binary_data` in `juce/CMakeLists.txt`. Aggiungere un `.nampreset` in `juce/Presets/Factory/` richiede **anche** l'aggiunta alla lista `SOURCES` del binary data — altrimenti il file non viene incluso.

Gain-staging (porting Steve)

Tre parametri APVTS controllano la modalità di output:

- `output_mode` (Raw / Normalized / Calibrated, default Normalized)
- `calibrate_input` (bool, applica correzione input basata su metadata)
- `input_cal_level` (float, dBu, default 12.0)

Logica:

Modello ha	Modalità richiesta	Comportamento
loudness + input/output_level	Calibrated	Correzione pre-model + post-model in dBu
solo loudness	Normalized	Compensazione loudness ~14 dBu di headroom
loudness + input_level (metà)	Normalized	Fallback 50/50: metà correzione pre-model, metà post-model

Modello ha	Modalità richiesta	Comportamento
nessun metadata	qualunque	No-op — il modello esce al suo livello nativo

Motivo del fallback 50/50: senza `output_level_dbu` non sappiamo dove sta il DAC virtuale del modello; splittare la compensazione evita saturazione dell'IR e self-noise amplificato a valle.

Doppio IR

Ogni `NAMPipeline` tiene **due** `IRConvolver`. Entrambi ricevono lo stesso ingresso e scrivono in buffer separati: se se ne usasse uno solo, il primo sovrascriverebbe il segnale prima che il secondo lo legga.

```
wet = (1-bal)·gain1·IR1(x) + bal·gain2·IR2(x)
out = x·(1-mix) + wet·mix
```

Dettagli che contano:

- con **un solo** IR presente il bilanciamento non attenua, altrimenti ruotando il pomello si perderebbe volume senza motivo
- quando il secondo è abilitato viene elaborato **anche a peso zero**, così la sua coda resta viva e ruotare il bilanciamento non produce scalini
- i picchi per IR sono azzerati a monte dello stadio, così in bypass i meter scendono invece di restare fermi
- `ir2_enable` viene acceso dal processore tramite un listener su `channel_mode` quando si passa a Dual-Mono o Stereo; tornando in Mono **non** viene spento

Esclusione automatica della cassa

`NAMPipeline::loadModel` legge `gear_type` dai metadata del modello via `GetMetadata`. Il valore arriva come JSON grezzo, quindi fra virgolette; si cercano le radici `cab` e `rig` per coprire le varianti (`amp_cab`, `full-rig`, `full_rig`, `amp+cab`). I valori osservati sul campo sono `amp`, `pedal`, `amp_cab`, `full-rig`.

Quando `modelHasCab()` è vero, l'editor forza `ir_bypass` a 1 e disabilita i comandi del primo loader. Tornando a un modello senza cassa il bypass viene riportato **com'era prima** dell'esclusione automatica, non forzato a zero.

Pedali selezionabili

Sei sezioni — `NGATE`, `GATE`, `COMP`, `OVERDRIVE`, `DISTORTION`, `EQ` — non hanno effetti fissi: sono **posti** nella catena, e cosa ci suona lo decide la tendina della sezione. Tre file soli:

Gli elenchi sono due: **`PedalRegistry.h`** per le sei sezioni della scheda `MAIN` e **`FxRegistry.h`** per le cinque della scheda `FX`, con `FxDSP.h` accanto. Stessa forma, stessi tipi di comando, contenuto diverso — e riserve di lunghezza diversa, otto comandi per il registro principale e cinque per quello `FX`, che non ha modelli così ricchi. `ModelView` nell'editor appiattisce i due registri in una vista sola, così il resto del codice non deve sapere da quale elenco venga il pedale che sta mostrando.

Un effetto in una sezione sola. Scegliendo a mano un modello già presente in un'altra sezione della stessa famiglia, la scelta viene rifiutata: `slotWithModel()` lo trova, il parametro torna al valore di prima e compare un avviso che dice dove sta già. Il controllo scatta solo per le scelte fatte a mano: uno stato salvato o un preset si caricano com'è, anche se contenessero un doppione. Il modello neutro "Bypass / Clean" è esente, perché non è un effetto.

- **`PedalRegistry.h`** — l'elenco, uno e condiviso. Ogni `Model` dichiara `id`, `nome`, `categoria`, **topologia**, una descrizione e la sua tabella di pomelli (`label`, `min`, `max`, `def`, `suffisso`) e interruttori. Le due costanti `kMaxKnobs = 8` e `kMaxSwitch = 1` dimensionano la riserva: otto perché il grafico a sette bande più il livello è il modello con più controlli, uno perché nessun pedale dell'elenco usa due interruttori.

- **PedalDSP.h** — gli algoritmi, senza dipendenze da JUCE, un'istanza per canale. `process()` smista sulla topologia, non sull'id: un primo tentativo distingueva i modelli dal quarto carattere dell'id e `od_screamer` e `od_super` finivano nello stesso ramo.
- Nell'editor, `pedalSlotFor()` mappa il nome della sezione sullo slot e `refreshPedalSection()` rietichetta la riserva quando cambia il modello.

Cosa appartiene allo slot e cosa al pedale. Allo slot appartengono la posizione nella catena, il parametro di bypass e il selettore Front / Post-Gate / Post-IR: sono proprietà del posto, non di chi lo occupa.

Come si è lasciato un pedale – valori dei pomelli, interruttore e accensione – e' invece del pedale, non della sezione. `modelMemory_` lo tiene per **modello** e non per slot, perché deve funzionare anche quando il pedale viene prima tolto di dove stava e poi ripreso altrove: al momento in cui lo si richiama, la sezione di partenza non lo ha più. I valori memorizzati sono quelli normalizzati 0..1 della riserva, quindi si spostano fra sezioni senza conversioni.

`applyRememberedState()` scrive nello slot solo se quel modello ha una memoria; altrimenti la sezione tiene la sua accensione e i pomelli partono dai valori di fabbrica del pedale.

Quale copia comanda. Con lo stesso pedale in due sezioni, la memoria deve seguire quella che l'utente sta regolando, non l'ultima che capita nel ciclo: `slotSeen_` tiene l'istantanea di ogni sezione al giro precedente e solo la sezione che è effettivamente cambiata aggiorna la memoria del modello. Senza questo, una copia ferma con i valori vecchi sovrascriveva quella appena regolata, e il pedale si ritrovava altrove com'era prima delle modifiche.

Chi ha chiesto il cambio. Riscrivere i comandi ha senso solo se il pedale l'ha scelto l'utente. `PedalBox` sovrascrive `showPopup()`, `keyPressed()` e `mouseWheelMove()` per marcare l'interazione: quando il cambio arriva dal ripristino di uno stato o dal caricamento di un preset, i valori giusti sono quelli appena caricati e riscriverli li perderebbe. Prima di questa distinzione, aprire la finestra riportava i pomelli dei pedali ai valori di fabbrica, e lo stesso succedeva caricando un preset con l'interfaccia aperta.

La torre di tono nativa è l'unico modello che non si duplica: i suoi comandi sono i pomelli nativi dell'equalizzatore, che sono componenti singoli e non possono stare in due pannelli. `pedalKnobIds()` glieli dà solo nello slot EQ; altrove la sezione resta senza comandi e mostra una riga che dice dove si regola, invece di un pannello vuoto senza spiegazione. Al pedale appartengono il titolo dipinto e il nome sul tasto (`pedalSectionTitle()`, la categoria in maiuscolo), i comandi, e i due accessori che lo seguono di sezione in sezione: l'analizzatore di spettro dell'equalizzatore e il misuratore di riduzione del compressore. Di questi ce n'è **uno per slot**, e si mostra quello dello slot che ospita il pedale giusto; il timer di ognuno parte in `visibilityChanged()`, così i nascosti non costano nulla.

`EQAnalyserComponent::showsCurve()` rilegge il parametro del modello invece dell'indice memorizzato: è quella funzione a decidere se l'analizzatore si mostra, e quindi se il suo timer parte — con un valore aggiornato dal timer non partirebbe mai.

La presa dello spettro segue l'equalizzatore. `NAMPipeline::setScopeSlot()` dice dopo quale slot campionare; il processore lo calcola dal primo slot che ospita un equalizzatore e ripiega sullo slot EQ. Lo slot del compressore può trovarsi in tre punti della catena, quindi la presa sta nel ciclo pre-modello o in quello post-IR a seconda del selettore di posizione. Se il blocco fosse più lungo del buffer allocato in `prepare()` la presa si salta: allocare sul thread audio non si può, e uno spettro che perde un blocco non si nota.

Perché una riserva di parametri. I parametri di un `AudioProcessorValueTreeState` si creano nel costruttore e non si possono aggiungere a runtime, mentre il pedale si sceglie mentre il plugin suona. Ogni sezione ha quindi <prefisso>_p1..p8 normalizzati 0..1 e <prefisso>_sw1, e il modello scelto dice come leggerli: il processore riporta ogni pomello nell'intervallo reale dichiarato ($k.min + norm * (k.max - k.min)$) prima di passarlo al DSP, e l'interfaccia fa la strada inversa in `textFromValueFunction`, altrimenti sui pomelli comparirebbe 0.313 invece di 120 ms.

Costo: **nove parametri per sezione** nella scheda MAIN e **sei** in quella FX — otto o cinque pomelli e un interruttore — più quello che sceglie il modello. È il minimo che questo schema consente: i parametri si creano nel costruttore, quindi la riserva deve bastare al modello più ricco, e da quando un pedale può andare in qualunque sezione la riserva dev'essere uguale in tutte.

Quello che si poteva togliere è stato tolto: i quattordici parametri nativi delle sezioni diventate slot

(`od_drive`, `ng_threshold`, `comp_sustain` e `compagnia`) restavano registrati e non li leggeva più nessuno, e con loro sparivano altrettanti pomelli costruiti e mai mostrati. Il secondo interruttore per sezione non lo usava nessun modello. I preset di fabbrica sono così passati da 186 a **166 parametri**, e poi a **185** quando le cinque sezioni FX hanno preso la loro riserva al posto dei sedici parametri nativi che avevano prima, che è esattamente quanti ne registra il processore: nessun parametro senza valore, nessun valore senza parametro.

Un preset salvato prima di questa pulizia si carica lo stesso: `replaceState` ignora i nodi che non corrispondono più a un parametro.

Topologie. Seguono gli schemi pubblici dei circuiti: clipping morbido **nell'anello** di reazione (famiglia `overdrive`, guadagno (R_f/R_g)+1), clipping duro **verso massa** dopo lo stadio di guadagno (famiglia `distorsore`), condensatore in serie a R_g da cui il passa-alto interno all'anello intorno ai 720 Hz. Sui distorsori è la **soglia** a decidere il timbro, non il guadagno: passato il ginocchio la forma d'onda è già piatta, e alzare solo il guadagno non cambia niente di udibile.

Compressori. Rilevatore di picco veloce (attacco 1 ms, rilascio 120 ms) separato dal livellatore del guadagno, che invece segue l'attacco impostato: è quello che lascia passare il transiente della pennata prima di stringere. Ginocchio morbido di 6 dB. Nei due sustainer il sustain muove insieme soglia (-6 ... -36 dB) e rapporto (2 ... 8 : 1); il limitatore espone soglia, rapporto e rilascio direttamente. `PedalFX:gainReductionDB()` alimenta il misuratore della sezione, e vale 0 quando il modello scelto non è un compressore.

L'equalizzatore. Il primo modello della categoria, `eq_tonestack`, è un passante: la torre di tono nativa è già nella catena e continua a fare il suo lavoro. Scegliendo un altro modello il processore **azzerà** le bande native, altrimenti le due curve si sommerebbero.

MAUSE RECTIFIER

`RectifierAmp.h`, stessa impostazione di `MarshallAmp.h`: header-only, mono, niente JUCE, stadi a triodo con $\tanh$ asimmetrica e torre di tono a biquad RBJ. Due canali con comandi indipendenti e tre modi ciascuno.

Il cedimento dell'alimentazione. Col raddrizzatore a valvole la tensione scende in qualche milisecondo sotto il segnale forte e risale in un paio di decimi: è quello che rende elastico l'attacco. Coi diodi resta ferma. Bold e Spongy non cambiano solo un guadagno: Spongy parte da una tensione più bassa, cede fino a poco più della metà e risale col doppio del tempo. Misurato su un gradino forte: 0,25 dB di cedimento in Bold, 1,81 dB in Spongy, -0,12 dB coi diodi, cioè nessuno.

Dove sta la presenza. Sul canale uno è nell'anello di controreazione, quindi dopo il Master; sul canale due nel preamplificatore, prima dello stadio finale, che la comprime. Con la stessa manopola a +10 dB si misurano +8,4 dB a 5 kHz sul canale uno e +1,1 dB sul due: è la ragione per cui la presenza del canale rosso è descritta come più dolce.

I modi. Clean sono due stadi appena spinti; Vintage tre stadi con la seconda armonica in evidenza; Modern tre stadi più duri con un passa-alto fra il secondo e il terzo, che è quello che tiene pulite le basse sotto guadagno alto. Misurata sulla stessa nota: THD 0,275 / 0,357 / 0,695.

Le sei teste a riserva condivisa

`AmpRegistry.h` e `AmpDSP.h`, stesso schema dei pedali: i modelli dichiarano i loro comandi e tutti si dividono una riserva sola — ventiquattro pomelli e quattro tendine. Sei teste con parametri dedicati ne avrebbero voluti quasi novanta; così ne bastano ventinove, e la settimana non ne costa nessuno. Ogni comando dichiara anche a quale canale appartiene, e da lì l'interfaccia ricava le file e attenua i canali che non suonano.

La perdita d'inserzione della torre di tono. Una torre passiva butta via dai dieci ai venti decibel, e va messa: senza, il finale riceve un segnale già saturo e le regolazioni non si sentono quasi. Nella prima stesura un taglio di 6 dB sull'equalizzatore grafico si misurava come 1 dB, perché la saturazione lo ricomprimeva subito, e le tre posizioni di potenza suonavano identiche. Aggiunta la perdita, lo stesso taglio si misura a 3,3 dB e il cedimento del finale passa da 0,88 dB a dieci watt a 0,27 a novanta.

Il cedimento non si regola solo col tempo. Con lo stesso tetto per tutte le potenze si arriva comunque allo stesso fondo, e le posizioni suonano uguali: il tetto va scalato con la potenza, e il tempo di risalita con lui.

Analizzatore EQ

EQAnalyserComponent disegna spettro e curva sovrapposti.

- **Spettro:** FFT a 2048 punti, finestra di Hann, 24 fotogrammi al secondo. Il segnale viene prelevato da una coda circolare riempita in `NAMPipeline::process` **subito dopo lo slot che ospita l'equalizzatore**, quindi si vede l'effetto della curva anche spostandolo in testa alla catena. Picco per bin con salita immediata e discesa lenta. Finestra utile $-100\dots-20$ dBFS: il livello del singolo bin è molto più basso di quello complessivo, e con 0 in cima lo spettro resta schiacciato sul fondo.
- **Curva:** dipende dal modello scelto nello slot. Per la torre di tono nativa è `FiveBandEQ::responseDB`, che costruisce biquad temporanei con **gli stessi setter della catena audio** e ne valuta la risposta; per i modelli a bande è la somma dei moduli delle campane RBJ, calcolati con gli stessi centri e Q che usa `PedalDSP`. In entrambi i casi la funzione è statica o pura e riceve i valori dei parametri: nessuno stato condiviso col thread audio, e la curva disegnata non può divergere da quella che si sente.
- **Maniglie:** la tabella delle bande non è fissa, la ricostruisce `rebuildBands()` quando cambia il modello dello slot. Cinque bande per la torre nativa (frequenza in orizzontale e Q sulla rotellina solo per la MID, l'unica parametrica), sette a frequenza fissa per il grafico, due spaziolabili per il parametrico. Quando la banda è comandata da un pomello della riserva, la maniglia converte fra dB e valore normalizzato in `bandParam/setBandParam`, così trascinare la maniglia muove il cursore della sezione e viceversa. I gesti sono racchiusi fra `beginChangeGesture` e `endChangeGesture`, così l'automazione dell'host registra un tratto unico.
- **L'analizzatore segue l'equalizzatore:** ce n'è uno per slot e si mostra quello dello slot che ospita un equalizzatore, non quello di una sezione fissa. Con un pedale che non è un equalizzatore la sezione non lo mostra affatto, e lo spazio torna ai comandi.

Con guadagno MID a 0 dB la risposta è esattamente 0.00 dB a ogni frequenza per qualunque Q: un filtro a campana senza guadagno è un passa-tutto. Non è un difetto del disegno.

Metronomo

`nam_dsp::Metronome` genera un click da campanaccio: due parziali inarmoniche che decadono insieme più una punta di seconda armonica per l'attacco. Il primo movimento della battuta sale di intonazione e dura di più (835/1235 Hz, decadimento 85 ms) rispetto agli altri (587/845 Hz, 45 ms).

Si mescola **dopo** il downsampling, quindi gira alla frequenza base e non a quella sovracampionata. Una bandiera atomica per movimento alimenta il lampeggio del TAP nell'interfaccia.

Modelli A2 (SlimmableContainer)

Rilevati via `NeuralAudio` (version > 0.5.4 o architecture == "SlimmableContainer" nel JSON del modello). Espongono:

- `HasQualityScaling()` — bool
- `SetQualityScaleFactor(float 0..1)` — RT-safe, pushata dall'editor via APVTS `quality_scale`

Lo slider SLIM sotto il loader NAM e il pomello QUAL sono legati allo stesso parametro; un `juce::Timer` a 8 Hz abilita o disabilita i due controlli in base a `isCurrentModelSlimmable()`.

`quality_scale` sceglie il sottomodello **per soglia**: `ContainerModel::get_index_for_slimmable_size` scorre i submodel e prende il primo con `max_value` superiore al valore. Con due submodel a soglia 0.5 e 1.0, un valore sotto 0.5 seleziona il più leggero. **Il default è 0**, quindi si parte dal più leggero.

Attenzione a due cose:

- non tutti gli A2 sono slimmabili: un WaveNet v0.6.0 non lo è, mentre lo sono sia Slimmable-Container sia il WaveNet slimmabile v0.7.0
- `setEnabled(false)` in JUCE rende un widget inerte ma **continua a disegnarlo acceso**: vanno attenuati anche i cursori, non la sola etichetta, altrimenti sembra abilitato lo stesso

IR loader

- Formato: WAV mono o multicanale (viene tenuto solo il canale 0).
- Sample rate sorgente: bounded ($0 < \text{srcRate} \leq 768 \text{ kHz}$ per rifiutare header forgiati).
- Cap frame: $\min(\text{totalPCMFrameCount}, \text{srcRate} \times 4)$ prima di allocare (evita OOM da header malevolo).
- Cap finale IR: $\sim 1.5 \text{ s}$ @ target sample rate.
- Lettura via `drwav_init_file` streaming (non slurp completo), `drwav_uninit` garantito su ogni path (incluso `bad_alloc`).

FX chain — implementazione

Tutti gli FX in `juce/PreampFX.h` sono classi **header-only, framework-independent** (nessun include JUCE). Ogni classe segue lo schema:

```
struct XxxFX {
    void prepare(double sampleRate, int blockSize); // alloca qui
    float process(float x);                       // hot path, alloc-free
};
```

Nuovi effetti vanno aggiunti nello stesso stile (memoria team `nam-juce-preampfx-framework-independent`).

Build

Requisiti Linux: `cmake` ≥ 3.15 , GCC/Clang C++17, ALSA, X11.

```
git clone --recurse-submodules https://github.com/fabionet/metal-nam-gear-player.git
cd metal-nam-gear-player
cmake -B build-juce -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build-juce -j 2
rsync -a --delete "build-juce/juce/NAMCustom_artefacts/Release/VST3/NAM Custom.vst3/" "$HOME"
```

Note:

- **-DCMAKE_BUILD_TYPE=Release obbligatorio**: senza flag esplicito il tipo di build è vuoto → binario non ottimizzato → audio a scatti.
- **-j 2 massimo su macchine con poca RAM**: `-j` illimitato può causare OOM-kill (exit 137).
- Il target `NAMCustom_LV2` produce `Metal NAM Gear Players Neo.lv2/`, il target `NAMCustom_Standalone` produce l'app JACK/ALSA.
- **Gli oggetti di NeuralAudio vanno aggiunti ai target finali**. È una OBJECT library i cui oggetti confluiscono nell'archivio `SharedCode` di JUCE; linkando quell'archivio il linker scarta i membri di cui nessuno referencia un simbolo, e i registratori statici delle architetture NAM non girano mai. Il sintomo è `No config parser registered for architecture: WaveNet` al caricamento di qualunque modello.

Pacchetto Debian

```
./packaging/linux/build-deb.sh 0.2.0
```

Assembla `metal-nam-gear-players-neo<ver>_amd64.deb` da un `build-juce` completato: VST3 in `/usr/lib/vst3`, LV2 in `/usr/lib/lv2`, autonomo in `/usr/lib/<pkg>` con collegamento in `/usr/bin`, voce di menu, licenze e guide.

Identità del plugin

	Full	Neo
Prodotto	Metal NAM Gear Player	Metal NAM Gear Players Neo
Codice VST3	...47313830	...4E67704E (NgpN)
URI LV2	urn:fabionet:metalnamgearplayer	urn:fabionet:metalnamgearplayers-neo
Pacchetto	metal-nam-gear-player	metal-nam-gear-players-neo

Il codice VST3 è l'identificatore su cui gli host distinguono i plugin: toccando PRODUCT_NAME, PLUGIN_CODE, BUNDLE_ID o LV2URI va verificato che restino diversi da quelli della Full.

Sicurezza (dalla v0.1.x)

- .nam corrotti/malevoli: nlohmann::json throw catturato → std::terminate dell'host evitato.
- WAV IR forgiati: cap frame + srcRate limitato + reject channels==0/>64 → OOM DoS mitigato.
- Use-after-free UI/audio in setOversamplingEnabled e clearModel/clearIR → risolti con suspendProcessing.
- **UNC path bypass (corretto nei binari dalla v0.2.0)**: isLocalSafePath rifiuta i percorsi UNC in entrambe le forme, i percorsi device NT \\?\ e \\.\, e gli schemi di rete smb: nfs: afp: ftp: http: https: cifs: dav: davs: file:. Il controllo si applica al modello, all'IR e al **secondo IR**.
- **Importazione preset**: solo le nostre estensioni, radice verificata, e i nomi degli asset con separatori o . . scartati per impedire scritture fuori dalla cartella.

Licenze

- **Opera combinata: AGPL-3.0-or-later.** L'upstream mikeoliphant/neural-amp-modeler-lv2 è GPL-3.0, ma JUCE 8 è **AGPLv3 / commerciale**: per GPL-3 §13 il derivato combinato viene distribuito AGPL-3.0-or-later (upgrade permesso, downgrade no).
- Motore **NeuralAudio**: MIT (compatibile).
- Framework **JUCE 8**: AGPLv3 / commerciale (Raw Material Software Ltd.).
- **VST3 SDK**: dual GPLv3 / proprietary Steinberg; usato sotto il ramo GPLv3 (compatibile con l'AGPLv3 risultante).
- Asset preset demo bundled: MIT.

Trademark: "JUCE" è marchio di Raw Material Software Ltd., "VST" di Steinberg Media Technologies GmbH, "Neural Amp Modeler" di Steven Atkinson. Il progetto non è affiliato con nessuno dei suddetti.

Contribuire

Issues e PR su <https://github.com/fabionet/metal-nam-gear-player>. Per il submodule NeuralAudio con le patch delle Has-flags, PR contro <https://github.com/fabionet/NeuralAudio> branch nam-custom-patches.